

Grade 9 Engineering & Autonomy

Robots That Work

20

SESSIONS

2 h

EACH · 40 H TOTAL

mBot2 Ranger

TANK + ROBOTIC ARM

Full Python

CODING MODE

★ Final project: Robots at Work

An autonomous mission simulating a real job: navigate by line, detect a marker, pick an object with the arm and deliver it — pitched and scored at Demo Day.

Grade 9 at a Glance

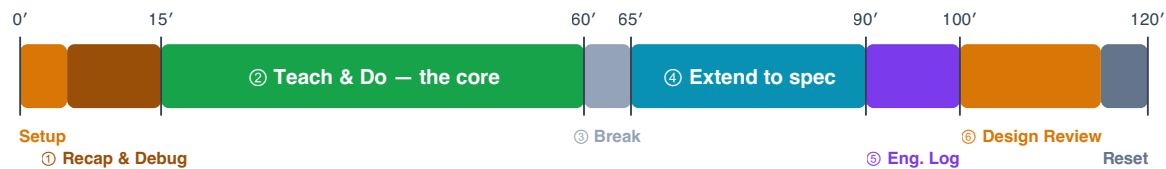
Learning objectives — by the end of the year, students can:

- Write, structure and review Python: functions, constants, state machines
- Implement and tune proportional control; argue with graphs
- Build the Ranger arm; program safe, reliable pick & place
- Run an engineering project: spec, sprints, reviews, recovery
- Deliver an autonomous mission and pitch it (Robots at Work)

The 20 sessions, colored by unit * in session titles = buffer session (tolerates compression if the schedule slips)



Every session runs on the same rhythm



Non-negotiables for 2-hour sessions: the engineering log is written during the session, not afterwards — a change with no measurement beside it does not count · every Extend task states its tolerance before anyone drives · design review is against the stated criteria, not taste · robots stay assembled on a labeled cart shelf and charge via USB-C between sessions.

Known friction points this year: the Ranger build guide is digital-only — download before Session 8 · never force the arm past resistance (gears strip) · exact Python function names vary by mBlock version: trust autocomplete · rebuild capstone courses from tape-measure photos.

The Boxes — Official Contents Reference

Both boxes are used this year. Session parts panels tag every piece with its source box.

BOX 1 · “mBot2”

CyberPi ×1	Ultrasonic Sensor 2 ×1	M4×8 mm screw ×6
mBot2 Shield ×1 · battery built in	Quad RGB Sensor ×1	M2.5×12 mm screw ×4
Chassis ×1	Motor cable ×2	Line-following track map ×1
Encoder motor ×2	mBuild cable 10 cm ×2	USB cable ×1
Wheel hub ×2	mBuild cable 20 cm ×1	Screwdriver ×1
Slick tyre ×2	M4×25 mm screw ×6	
Mini wheel ×1	M4×14 mm screw ×6	

Quantities from the official packing list. A Bluetooth dongle is NOT included and this program does not use one — it is only needed by computers with no built-in Bluetooth, and USB pairing is the documented fallback in every session that connects a robot.

BOX 2 · “Rover Robotics Add-on Pack”

Servo pack ×2 used	Slide beam 2×4 ≥×1	Nut ≥×30
Bluetooth Controller ×1	Slide beam 2×6 ≥×1	Plastic shock absorber spares — unused in build
Track ×2	Plate 3×8 ≥×1	Metal shock absorber ×4 used
Wheel hub ×4 halves used	Shaft collar ×12 used	Long plastic ring ≥×2
Small wheel hub ×2 used	Flange bearing ≥×4	Short plastic ring ≥×8
D shaft ≥×6	Socket wrench ×1	Rubber band ≥×6
Bracket 90° 3×3 ≥×6	Screw M3×6 mm ×12 used	Board 1–8 (body shells) peel paper film
Bracket 120° 3×3 ≥×3	Screw M4×10 mm ≥×30	Quick Start Guide ×1
U bracket ≥×4	Screw M4×22 mm ≥×14	
T plate ×2 used	Screw M4×30 mm ≥×8	

As printed on the box (no quantities given). Online docs: search “mBot2 Rover” at support.makeblock.com.

Same Robot, New Language

Goal of these two hours: Both partners write and fix Python unaided, and an error message is read as information rather than as a wall.

YOU NEED — PIECES & MATERIALS



Rover,
assembled
from Grade 8

Rover add-on
box



Tablet + mBlock
Web
mBlock Python
mode

Classroom



Engineering
journal
error-message
notes

Classroom

TEACH IT — THE CONCEPT

Open with the Grade 8 fair photos, then the year's premise: same robot, professional tools. mBlock's Python mode is a real editor — text, autocomplete, error messages. The first program is deliberately familiar: the square. They've built it in blocks (Grade 7) and read it in Python (Grade 8, Session 17); today they type it from a blank file. Familiar problem + new medium = confidence, not overwhelm.

Institute **pair programming** as this year's standard: driver types, navigator reads ahead and catches errors; swap every 10 minutes. It's how professionals work and it halves the syntax frustration of week one. Also set the error-message culture: an error message is information, not an insult — read it aloud, bottom line first.

② TEACH & DO — THE CORE (45')

- 8'** Editor tour: file, run, stop, console, autocomplete. Break something on purpose; read the error together.
- 20'** Pairs type the square from scratch; run on the robot; swap roles halfway.
- 12'** Extension ladder: triangle → hexagon → any-N polygon (the 360/N insight, now as code).
- 5'** Journal: one error message you met and what it was telling you.

PYTHON (MBLOCK → PYTHON MODE)

```
import cyberpi
import mbot2

# our old friend, typed from scratch
for i in range(4):
    mbot2.straight(30)    # cm forward
    mbot2.turn(90)       # degrees right

cyberpi.display.show_label('done!', 24,
                           'center')
```

Watch out

- Function names below follow mBlock's CyberPi Python API; minor names vary by version — the class rule is: trust the editor's autocomplete.
- IndentationError and missing colons will dominate today — predict it aloud so nobody feels alone.
- Pairs stay together for the whole Python Bootcamp unit.

+ THE 2-HOUR SESSION — THE OTHER 65' (2 TEACH & DO = THE 45' CORE ABOVE · 3 BREAK 5')

① RECAP & DEBUG

Build an error museum. Each pair deliberately produces three different errors — a missing colon, a bad indent, an undefined name — and writes down what each message actually says. Collect them on the board. A class that has met the three commonest messages on purpose will not lose twenty minutes to them in week four.

⑤ ENGINEERING LOG

An error page — message, what it means, what fixed it. It grows all year and becomes the fastest debugging asset the class owns, because the second time a message appears it costs thirty seconds instead of ten minutes.

10' ④ EXTEND — TO SPEC

any-N polygon, typed One function, three shapes, written as text rather than dragged.

roles swapped, both fluent The navigator takes the keyboard and the pair's speed does not collapse. If it does, the navigator was watching rather than reading.

3 errors decoded Three different messages read, explained in the pair's own words, and fixed.

10' ⑥ DESIGN REVIEW

Pairs teach one error message to a pair that has not met it. Criteria — is the explanation about what Python was trying to say, or is it "add a colon here"? Only the first transfers to the next error.

25'

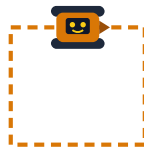
15'

IF YOU FINISH EARLY **Constrain it** Rewrite your polygon program in the fewest lines you can manage while keeping it readable to your partner — then have them read it back to you. Shortest is not the goal; shortest-that-still-reads is.

• EXPECTED RESULT AT THE END OF THIS SESSION



typed Python drives the same square as Grade 7's blocks



- ☒ Square runs from hand-typed Python
- ☒ Pair-programming roles practiced with a swap
- ☒ At least one error message read, decoded, and fixed by the pair
- ☐ The navigator produced working code after the swap
- ☐ Three error messages explained in the pair's own words

Functions

Goal of these two hours: A function library another pair can use cold, with no explanation offered and no questions answered.

YOU NEED — PIECES & MATERIALS



Rover,
assembled

Rover add-on
box



Tablet + mBlock
Web

Classroom



Engineering
journal

library design notes

Classroom

TEACH IT — THE CONCEPT

A function is a named behavior: define `square(size)` once, call it anywhere, change it in one place. Connect backwards: functions are to code what loops were to repeated blocks in Grade 7 — the next rung on the same ladder (don't repeat yourself). The parameter is the power move: `square(20)`, `square(40)` — one definition, infinite squares.

Build a small personal library today: `square(size)`, `polygon(n, size)`, `celebrate()` (LEDs + sound). This library literally gets imported into the capstone — tell them that now, so “write it well” has a reason. Naming matters: `celebrate()` reads like English; `func2()` reads like regret.

② TEACH & DO — THE CORE (45')

- 10'** Live-code `square()` together; call it three times with different sizes.
- 15'** Pairs build `polygon(n, size)` — the 360/N rule becomes one line of math in code.
- 12'** Design `celebrate()` freely: lights + sound + a wiggle; must be a single call.
- 8'** Library review: swap code with another pair; can they use your functions without asking you?

PYTHON (MBLOCK → PYTHON MODE)

```
import mbot2, cyberpi

def polygon(n, size):
    for i in range(n):
        mbot2.straight(size)
        mbot2.turn(360 / n)

def celebrate():
    cyberpi.led.on(255, 180, 0)
    cyberpi.audio.play('yay')

polygon(5, 25) # pentagon, one line
celebrate()
```

Watch out

- The another-pair-can-use-it test is the real definition of a good function.
- Watch for logic vs. syntax confusion: `polygon(3, ...)` turning 120° is correct — remind them of the exterior angle.

+ THE 2-HOUR SESSION — THE OTHER 65' (② TEACH & DO = THE 45' CORE ABOVE · ③ BREAK 5')

① RECAP & DEBUG

Run the swap test properly. Hand your library to another pair with no README and no talking, then watch what they get wrong. Every misuse is a naming problem or a parameter problem, and both are visible within seconds — which is why this is the real definition of a good function, not a nice-to-have.

⑤ ENGINEERING LOG

The library with a one-line comment per function saying what it does and what the parameters mean. This code is imported by the capstone in Session 15, so it is being written for a reader fifteen weeks from now — and that reader is usually its author.

10' ④ EXTEND — TO SPEC

used cold, no questions Another pair calls your functions with no explanation given and none asked for.

polygon(n, size), 3 shapes One function, three different shapes, exterior angles correct in every case.

celebrate() as a single call Everything inside the function; nothing left over at the call site.

10' ⑥ DESIGN REVIEW

15' The receiving pair reports what they got wrong. Criteria — was the confusion caused by a name or by a parameter? Both are free to fix today and expensive to fix during a build sprint.

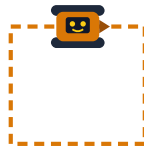
IF YOU FINISH EARLY **Bank it** Add one more function you will genuinely want in the capstone, and write its one-line comment *before* the body. If you cannot write the comment, the function is not yet a single idea and should be split.

• EXPECTED RESULT AT THE END OF THIS SESSION

```
def polygon(n, size):
    for i in range(n):
        mbot2.straight(size)
        mbot2.turn(360/n)

polygon(4, 30)
```

one function, many shapes — the personal library begins



- ☒ polygon(n, size) draws at least three different shapes
- ☒ celebrate() runs as a single call
- ☒ Another pair used the library without explanations
- ☐ Another pair used the library cold, with no explanation
- ☐ Every function carries a one-line comment aimed at a future reader

Sensing in Python

Goal of these two hours: The line offset understood physically — its sign and rough size predicted before the robot is pushed anywhere.

YOU NEED — PIECES & MATERIALS



Rover,
assembled

Rover add-on
box



Tablet + mBlock
Web

Classroom



Line-following
map
line for the offset
explorer

mBot2 box



Ruler / tape
measure
verify distances

Classroom



Engineering
journal
offset sketch

Classroom

TEACH IT — THE CONCEPT

Sensors in Python return numbers you can hold in a variable: `d = cyberpi.ultrasonic2.get()`. That one line changes the relationship — in blocks, values flowed invisibly; in Python, students capture, print, compare and compute with them. Build the classic diagnostic tool first: a loop printing distance to the screen ten times a second (a live dashboard, and their debugging best friend for the rest of the year).

Then the line sensor: it reports how far off-center the line is (an **offset**, roughly $-100 \dots +100$). Have students push the robot along a line by hand and watch the number swing negative–zero–positive. That number IS next week's error signal for P-control; today they just need to feel what it measures.

② TEACH & DO — THE CORE (45')

- 12'** Build the distance dashboard; verify against a ruler; find sensor limits again (they know the drill).
- 12'** Threshold play: print 'NEAR!' when `d < 15` — an if on a captured value.
- 15'** Line offset explorer: hand-push the robot along a line, watch the offset swing; sketch its meaning.
- 6'** Journal: what does offset = -60 mean physically? Draw it.

PYTHON (MBLOCK → PYTHON MODE)

```
import cyberpi, mbot2, time

while True:
    d = cyberpi.ultrasonic2.get()
    off =
    cyberpi.quad_rgb_sensor.get_offset_track()
    cyberpi.display.show_label('d=' +
    str(d), 16, 'top')
    cyberpi.display.show_label('off=' +
    str(off), 16, 'center')
    if d < 15:
        cyberpi.display.show_label('NEAR!',
        24, 'bottom')
    time.sleep(0.1)
```

Watch out

- Function names below follow mBlock's CyberPi Python API; minor names vary by version — the class rule is: trust the editor's autocomplete.
- `str(d)` trips everyone — numbers must become text to display; let them hit the error, then explain.
- Calibrate the line sensor first (the Grade 7 ritual) or the offset numbers are noise.

+ THE 2-HOUR SESSION — THE OTHER 65' (② TEACH & DO = THE 45' CORE ABOVE · ③ BREAK 5')

① RECAP & DEBUG

Predict the number first. Before pushing the robot along the line, each pair writes what the offset will read in three positions: line hard left, dead centre, line hard right. Then push and check. Getting the sign backwards is common, and it is far cheaper to discover here than in Session 6, where an inverted sign makes the robot flee the line.

⑤ ENGINEERING LOG

The sign convention in a box in the journal — which way is positive — plus the three predicted and actual offsets side by side. Session 6 reads this box, so a convention recorded vaguely costs a session's tuning to an inverted term.

10' ④ EXTEND — TO SPEC

2 values live Distance and line offset both updating on the dashboard at once.

±1 cm against a ruler The distance reading verified as in Grade 7 — the drill transfers, and so does the habit.

offset explained physically What offset = -60 means, drawn, with the sign convention written down explicitly.

10' ⑥ DESIGN REVIEW

15' Pairs explain their sign convention to another pair. Criteria — do the two pairs agree, and where they do not, is it a genuine difference in how the sensor is mounted or simply a drawing error?

IF YOU FINISH EARLY **Break it** Find a position where the offset reads zero but the robot is not centred. There is usually one — a line edge, a shadow, a join in the floor. Record it, because Session 6's follower will believe that zero completely.

• EXPECTED RESULT AT THE END OF THIS SESSION

```
while True:
    d = ultrasonic2.get()
    show('d=' + str(d))
    if d < 15:
        show('NEAR!')
    time.sleep(0.1)
```

a live sensor dashboard — the year's debugging companion

d = 34 cm
d = 27 cm
d = 12 cm
STOP

- ☒ Dashboard shows live distance + line offset
- ☒ NEAR! fires at the right threshold
- ☒ Team can explain offset's sign and rough magnitude physically
- ☐ Offset sign predicted correctly before the robot was pushed
- ☐ The sign convention written where Session 6 can find it

Logic & Refactor

Goal of these two hours: Refactored code whose behaviour is proved unchanged by measurement rather than asserted by the people who changed it.

YOU NEED — PIECES & MATERIALS



Rover,
assembled

Rover add-on
box



Tablet + mBlock
Web

Classroom



Boxes /
obstacles
arena

Classroom



Engineering
journal
before/after snippets

Classroom



Stopwatch

Classroom

TEACH IT — THE CONCEPT

Full control flow: `if / elif / else` and `while`. The vehicle is deliberately nostalgic — rewrite Grade 7's wander mode in Python. They know exactly what it should do, so all attention goes to expressing it in the new language: a `while-True` loop, a distance check, the back-up-and-random-turn escape.

Then introduce **refactoring**: improving code's shape without changing its behavior. Extract the escape maneuver into `avoid()`, name the magic numbers as constants (`SAFE_CM = 12`), and re-run — identical robot behavior, radically clearer code. That “same behavior, better shape” discipline is the heart of code quality, and it's a rubric line in the capstone code review.

② TEACH & DO — THE CORE (45')

- 15'** Rewrite wander in Python: `while True` + distance check + escape.
- 15'** Refactor: extract `avoid()`, name constants, add two comments that say **WHY** not **what**.
- 10'** Arena test: does refactored wander behave identically? (It must — that's the definition.)
- 5'** Journal: before/after code snippets — which would you rather debug at 11 pm?

PYTHON (MBLOCK → PYTHON MODE)

```
import mbot2, cyberpi, random

SAFE_CM = 12          # escape trigger
distance
CRUISE = 40           # RPM

def avoid():
    mbot2.straight(-10)
    mbot2.turn(random.randint(60, 120))

while True:
    if cyberpi.ultrasonic2.get() < SAFE_CM:
        avoid()
    else:
        mbot2.forward(CRUISE)
```

Watch out

- `random.randint` needs the import — a good ‘read the error’ moment.
- ‘Why not what’ comments: `# escape trigger distance`, not `# set SAFE_CM to 12`.

+ THE 2-HOUR SESSION — THE OTHER 65' (② TEACH & DO = THE 45' CORE ABOVE · ③ BREAK 5')

① RECAP & DEBUG

Record the behaviour before touching it — three arena runs with the original code, timed, contacts counted. Without that baseline, "behaviour unchanged" is an opinion, and the whole point of the word refactoring is that the claim can be checked.

⑤ ENGINEERING LOG

Before-and-after measurements side by side, with the before-and-after code snippets beneath them. The journal question is the honest one: which of these two would you rather be debugging at 11 pm?

10' ④ EXTEND — TO SPEC

identical, measured The same three-run measurement after refactoring, landing inside the spread of the before.

0 magic numbers Every threshold a named constant, defined in exactly one place.

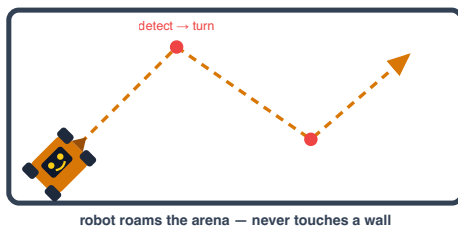
why, not what Two comments explaining intent, and no comment that merely restates the line beneath it.

10' ⑥ DESIGN REVIEW

15' Another pair reads the refactored version cold and states what the robot does. Criteria — could they say it without seeing the original, and did the named constants help, or did they just add a layer to chase?

IF YOU FINISH EARLY **Teach it** Hand your refactored file to another pair and ask them to describe what the robot does, without showing them the original. Whatever they get wrong is a naming problem, and naming problems are never cheaper to fix than today.

• EXPECTED RESULT AT THE END OF THIS SESSION



- ☒ Python wander roams the arena reliably
- ☒ avoid() extracted; constants named; behavior unchanged
- ☒ Comments explain why, not what
- ☐ Behaviour proved unchanged by measurement rather than by claim
- ☐ No magic numbers survive anywhere in the file

The Wobble, Explained

Goal of these two hours: The Grade 7 wobble question answered in control terms, with motor speeds computed by hand before a single line of code exists.

YOU NEED — PIECES & MATERIALS



Masking tape
taped line on the floor

Classroom



Blindfold
the control-theory theatre

Classroom



Engineering journal
hand computations

Classroom



Printed cards
printed offset scenarios

Classroom

TEACH IT — THE CONCEPT

Resurrect Grade 7's board question: *why does the line follower zig-zag?* Now they can answer it. Bang-bang control knows only three answers (hard left / straight / hard right) — it always overcorrects, overshoots, and corrects back: the wobble is built into the logic. The fix is one idea: **make the correction proportional to the error**. Slightly off → tiny steering nudge; far off → strong steering. In code it will be one line: `steer = error × gain`.

Teach it with the blindfold walk before any code: one student walks a taped line blindfolded; their partner may only say “left”/“right” (bang-bang) — watch them wobble. Then allow “a little left” / “way left” (proportional) — the walk visibly smooths. Ten minutes of theatre worth a thousand words. Then define terms on the board: **error** (the line offset they measured in Session 3), **gain** (how aggressively we respond), and predict together what too-low gain (drifts off curves) and too-high gain (wobbles again!) will look like tomorrow.

Watch out

- Do NOT code today — the concept needs its own hour; tomorrow's implementation then takes minutes.
- The hand computation ($\text{offset} \times \text{gain} \rightarrow \text{two motor speeds}$) is the comprehension gate; don't skip it.

② TEACH & DO — THE CORE (45')

- | | |
|-----|--|
| 12' | Blindfold line-walk, bang-bang vs. proportional instructions; debrief what changed. |
| 12' | Board the model: $\text{error} \rightarrow \times \text{gain} \rightarrow \text{steering}$; connect error to Session 3's offset number. |
| 14' | Paper-trace the algorithm on drawn scenarios: offset -60, gain 0.5 → motors? Teams compute by hand. |
| 7' | Predictions in the journal: sketch what gain-too-low and gain-too-high trajectories will look like. |

+ THE 2-HOUR SESSION — THE OTHER 65' (② TEACH & DO = THE 45' CORE ABOVE · ③ BREAK 5')

① RECAP & DEBUG

Walk it again, harder. The blindfolded partner now gets proportional instructions only — "a little left", "way left" — and the pair must agree in advance what "a little" means in steps. Quantifying the vague word *is* the gain, and discovering that is worth far more than being handed the term.

⑤ ENGINEERING LOG

The four computations with working, and the two predicted trajectories. Tomorrow the robot either matches these sketches or it does not — both outcomes are useful, and neither is available unless the sketches exist first.

10' ④ EXTEND — TO SPEC

25'

4 hand computations Offset and gain given, both motor speeds computed on paper, all four correct with working shown.

2 failure sketches Gain too low and gain too high, trajectories drawn before either is ever run.

the Grade 7 question, answered The wobble explained in one paragraph using error, gain and overcorrection.

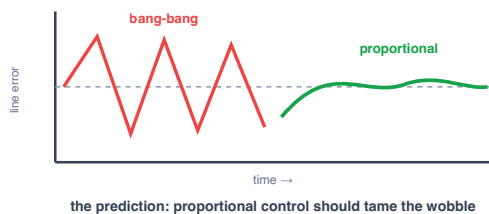
10' ⑥ DESIGN REVIEW

15'

Pairs present their answer to the board question that has been open since Grade 7. Criteria — does the explanation use error and correction, rather than "it's just jerky"? Two years of waiting has earned a real answer.

IF YOU FINISH EARLY **Constrain it** Do the blindfold walk again with only three words permitted: hard, little, straight. Fewer words means coarser control — and feeling that limit is exactly what a low gain does to a robot.

• EXPECTED RESULT AT THE END OF THIS SESSION



- ☒ Class answered the Grade 7 wobble question in control terms
- ☒ Teams hand-computed motor speeds from offset × gain correctly
- ☒ Both failure predictions (low/high gain) sketched
- ☐ Four motor-speed computations correct by hand, with no code written all hour
- ☐ Both failure trajectories sketched in advance of any run

P-Control

Goal of these two hours: A champion gain chosen from a table, and the gain at which the wobble returns found and checked against yesterday's prediction.

YOU NEED — PIECES & MATERIALS



Rover,
assembled

Rover add-on
box



Tablet + mBlock
Web

Classroom



Line-following
map
same track all laps

mBot2 box



Stopwatch
lap times

Classroom



Engineering
journal
gain table

Classroom



mBot2 Shield
same battery level
across laps

mBot2 box

TEACH IT — THE CONCEPT

Implementation day: read the offset, multiply by gain, split the result across the two tracks. The core is genuinely three lines — emphasize that a big idea can be small code. Then tuning: start with a low gain, raise it until the robot hugs curves, and keep going until it wobbles again (yesterday's prediction comes true — applaud it, it means the model is right).

Tuning protocol is Session 15/13 discipline: one gain value → one timed lap → log it → change once. Teams produce a small gain table (gain / lap time / observation) and pick their champion value.

Vocabulary upgrade: they are tuning a **control loop** — the same mathematics that keeps drones level and cruise control steady. They should know the thing they built today is not a toy idea.

PYTHON (MBLOCK → PYTHON MODE)

```
import mbot2, cyberpi

BASE = 45      # cruise RPM
GAIN = 0.5     # tune me!

while True:
    err =
    cyberpi.quad_rgb_sensor.get_offset_track()
    steer = err * GAIN          # the
    whole idea
    mbot2.drive_speed(BASE + steer, BASE -
    steer)
```

Watch out

- Function names below follow mBlock's CyberPi Python API; minor names vary by version — the class rule is: trust the editor's autocomplete.
- If the robot flees the line instantly, the offset sign is inverted — flip the +/- and move on.
- Same track and battery across tuning laps, or the table lies.

② TEACH & DO — THE CORE (45')

- 12'** Type the P-follower; first run at gain ≈ 0.3 — expect lazy curve-cutting.
- 22'** Tuning sprints: gain table with 5+ values, timed laps, one change at a time.
- 8'** Champion runs: best gain, three laps, best time recorded next to Grade 7's bang-bang times.
- 3'** Journal: your gain table + the value where wobble returned.

+ THE 2-HOUR SESSION — THE OTHER 65' (2 TEACH & DO = THE 45' CORE ABOVE · 3 BREAK 5')

① RECAP & DEBUG

Run yesterday's predictions. Set the gain deliberately too low, then deliberately too high, and compare the real trajectories against the sketches drawn when no code existed. Predictions that held are satisfying; predictions that failed are where the mental model needs repair, and that is the more valuable of the two.

⑤ ENGINEERING LOG

The gain table — gain, lap time, observation — champion value circled, wobble-return value marked. Same track and same battery across the whole table, or it lies; note the battery level so a future reader can tell whether it did.

10' ④ EXTEND — TO SPEC

5+ gains, one change each A gain table with timed laps and one change per row.

beat bang-bang both ways Smoother *and* faster than the Grade 7 times still sitting in the journal.

wobble-return gain found The value at which instability comes back, located and recorded rather than avoided.

10' ⑥ DESIGN REVIEW

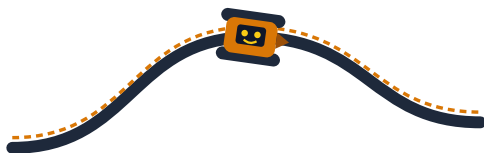
Pairs defend their champion gain. Criteria — is the choice supported by the table, and can they say what goes wrong on each side of it? A champion value with no neighbours tested is a lucky guess.

25'

15'

IF YOU FINISH EARLY **Break it** Find the gain at which the robot is unstable but still finishes the lap. That narrow band between smooth and wild is where most real control tuning happens, and it is worth seeing once with your own robot.

• EXPECTED RESULT AT THE END OF THIS SESSION



P-control: the trajectory finally hugs the line

- ☒ P-follower completes laps visibly smoother than bang-bang
- ☒ Gain table with 5+ tested values, one-change discipline
- ☒ Wobble-returns gain found, matching the prediction
- ☐ Wobble-return gain found and compared against the Session 5 prediction
- ☐ Champion gain defended from the table rather than from feel

Measure It

Goal of these two hours: A written conclusion that cites numbers from graphs the team produced out of its own logged data.

YOU NEED — PIECES & MATERIALS



Rover,
assembled

Rover add-on
box



Tablet + mBlock
Web

Classroom



Line-following
map

mBot2 box



Engineering
journal

graph paper
Classroom



Printed cards
or spreadsheet

Classroom



Laptop + mBlock
spreadsheet, or
paper grids

Classroom



Craft kit (card,
markers, tape)

Classroom

TEACH IT — THE CONCEPT

Engineers don't say "it looks smoother" — they show a graph. Today: log the error signal while driving (append offset values to a list, or print them and transcribe), then plot error-vs-time for bang-bang and for P-control. The bang-bang plot is a square wave bouncing between extremes; the P-control plot shrinks and settles. One picture, whole argument.

Pair it with lap-time data: P-control is usually both smoother AND faster — a rare free lunch, and worth noticing. Fast teams get the D-term teaser: error $\times$ gain still overshoots a little because it reacts to where the robot IS, not where it's HEADING; adding a term based on how fast error is changing (the derivative) damps that. Even implementing nothing, knowing PID exists — and that they built the P — is the doorway out of this course into real control engineering.

② TEACH & DO — THE CORE (45')

- 15'** Add error logging to both followers; drive 20 s each, capture the series.
- 15'** Plot both series (paper grid or spreadsheet); overlay or side-by-side.
- 10'** Compare + write the two-sentence evidence-based conclusion under the graphs.
- 5'** D-term teaser for all; extension pairs may prototype it.

PYTHON (MBLOCK → PYTHON MODE)

```
errors = []
for i in range(200):          # ~20 s of
    driving
        err =
        cyberpi.quad_rgb_sensor.get_offset_track()
        errors.append(err)
        steer = err * GAIN
        mbot2.drive_speed(BASE + steer, BASE -
        steer)
        time.sleep(0.1)

print(errors)                # copy out for
                              plotting
```

Watch out

- Transcribing 200 numbers is misery — sample every 5th, or screenshot the console.
- The conclusion sentence must cite the graph ('peak error fell from ~90 to ~25'), not vibes.

+ THE 2-HOUR SESSION — THE OTHER 65' (② TEACH & DO = THE 45' CORE ABOVE · ③ BREAK 5')

① RECAP & DEBUG

Sketch the shape before plotting the numbers. From memory of yesterday's driving, draw what the bang-bang error trace and the P-control trace should look like. Then plot the real data. A square wave against a damped curve is visible before any statistics happen — and predicting it first is what makes the plot mean something.

10'

④ EXTEND — TO SPEC

25'

2 series, real data Both followers logged over 20 s and plotted with axes and units.

conclusion cites numbers Peak error fell from X to Y; lap time from A to B. Named figures, taken from your own plot.

D-term in your words What a derivative term would add, in one sentence, without reciting the acronym.

⑤ ENGINEERING LOG

Both plots, and the two-sentence conclusion written underneath citing the figures. "It looks smoother" is not a conclusion — it is precisely the sentence the graph was made in order to replace.

10'

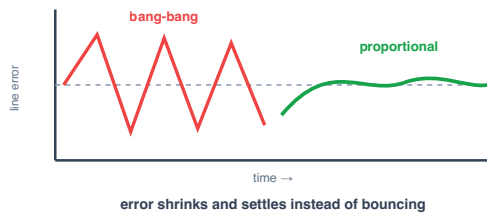
⑥ DESIGN REVIEW

15'

Teams challenge each other's conclusions. Criteria — is every claim traceable to a number on the plot, and does the axis actually measure what the claimant thinks it measures?

IF YOU FINISH EARLY **Reliability** Log a second run of the same follower at the same gain and overlay the two traces. If they do not look alike, your first graph was a story about one lap rather than about the controller.

• EXPECTED RESULT AT THE END OF THIS SESSION



- ☒ Both error-time plots produced from real logged data
- ☒ Written conclusion cites numbers from the graphs
- ☒ Team can say what the D in PID would add
- ☐ The written conclusion cites specific figures from the team's own plots
- ☐ The D-term explained in the team's own words, not recited

Ranger Build I

Goal of these two hours: A base rigid enough to carry a moving arm — proved by a twist test in someone's hands, not by looking at it.

YOU NEED — PIECES & MATERIALS

 Rover, assembled to tear down Rover add-on box	 Sorting trays the ritual Classroom	 Track x2 Rover add-on box	 Wheel hubs (large + small) Rover add-on box	 D shaft Rover add-on box	 Shock absorbers Rover add-on box	 Rover screws + nuts + socket wrench & screwdriver Rover add-on box	 Tablet + mBlock Web Ranger guide — DOWNLOADED Classroom
 Quick Start Guide downloaded before the session Rover add-on box	 Screwdriver mBot2 box	 Engineering journal Classroom					

TEACH IT — THE CONCEPT

The Ranger is the kit's final form: tracked base plus a servo-driven robotic arm. Today: structured teardown of the Rover (the Grade 8 tray ritual — they know it; hold them to it) and the tank base per the digital guide. Logistics note repeated from the plan: the Ranger guide is **digital-only** — have it downloaded, projected, and ideally printed before class; a dead link kills the hour.

Build quality bar is higher this year: the base will carry a moving arm, so torsional rigidity matters — loose frame screws that were tolerable on the Rover become arm-shake later. The QC checklist gains a line: twist-test the frame gently; nothing should creak or flex visibly.

② TEACH & DO — THE CORE (45')

- | | |
|------------|---|
| 12' | Rover teardown into trays; electronics padded; parts checked against the Ranger list. |
| 28' | Tank base from the guide: front unit (pp. 5–14), then chassis (pp. 15–29); frame screws snugged in a cross pattern. |
| 5' | QC + twist test; label robots; journal: what will the arm demand from this base? |

QUICK START GUIDE — THIS SESSION

- | | |
|-------------------|---|
| Guide | mBot2 Ranger Quick Start Guide (the digital ZIP, 61 printed pp.): front unit pp. 5–14 → chassis pp. 15–29. NOT the help-center's "mBot Ranger" — different, older kit |
| Screws | front unit opens x8+x8+x8; same fastener system as the Rover (M4×10/22/30 + nuts, socket wrench); 3 parts reused from the mBot2 box — read panel counts off the projected guide |
| Watch-outs | wheel screws: tighten, then slightly counter-clockwise so the wheel rotates · track grain direction, checked on BOTH sides |

Watch out

- Download the guide in advance — digital-only, and school wifi chooses violence on demo days.
- Track tension lesson from Grade 8 applies verbatim; fingerpress test again.

+ THE 2-HOUR SESSION — THE OTHER 65' (② TEACH & DO = THE 45' CORE ABOVE · ③ BREAK 5')

① RECAP & DEBUG

Twist the Rover before tearing it down, and feel where it flexes. That memory sets the standard for today's base, because the arm amplifies every millimetre of flex into shake at the tip. Teardown goes faster once the reason for building tighter is in the hands rather than on a slide.

10' ④ EXTEND — TO SPEC

25'

twist test, no creak The frame gently twisted with no creak and no visible flex.

cross-pattern, checked Frame screws snugged in a cross pattern and verified by another team.

guide offline The digital guide downloaded and openable with the wifi switched off. School networks choose violence on exactly the wrong day.

⑤ ENGINEERING LOG

Tray verification against the Ranger parts list, and the twist-test result signed by a checker. Note which screws needed a second pass — those are the ones to re-check once the arm is on and shaking them.

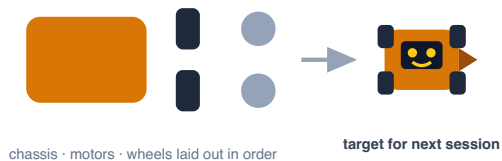
10' ⑥ DESIGN REVIEW

15'

Checker teams report on rigidity. Criteria — was the twist actually performed by hand, and did the checker find flex the builders had stopped noticing?

IF YOU FINISH EARLY **Teach it** Twist-test another team's base and sign for it. You know what a creak means now — and a checker who has never felt a good frame is only providing a signature.

• EXPECTED RESULT AT THE END OF THIS SESSION

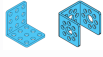
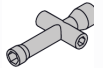


- ☒ Teardown complete, trays verified
- ☒ Tank base passes QC + twist test
- ☒ Ranger guide accessible offline for next session
- ☐ The base passes a hand twist test performed by another team
- ☐ The guide is available offline before anybody needs it

Ranger Build II

Goal of these two hours: The arm's safe range measured by hand for this particular robot, and the lever story told from the team's own sketch.

YOU NEED — PIECES & MATERIALS

 Servo pack the arm's muscle Rover add-on box	 Brackets 90° / 120° / U Rover add-on box	 Slide beams + plates arm linkage Rover add-on box	 Boards 1–8 gripper + mounts Rover add-on box	 Rubber bands grip traction Rover add-on box	 Rover screws + nuts + both drivers Rover add-on box	 Socket wrench Rover add-on box	 Engineering journal safe-range numbers Classroom
 Ranger, assembled Rover add-on box	 Protractor Classroom						

TEACH IT — THE CONCEPT

Assemble the arm: servo, linkage, gripper. Then teach the machine they just built. The arm is a **lever**: the servo twists at the joint, the load hangs at the tip — the longer the arm, the more **torque** the same load demands (why cranes are thick at the base and thin at the tip). If the design routes power through gears: the ratio trades speed for force, like bicycle gears — climbing gear vs. sprint gear.

Make it tactile: unpowered, let students feel the difference between pushing the arm near the joint vs. at the tip. Then the engineering consequence: servos have limits — stall a servo against an obstacle and it overheats. That physical fact becomes next session's software rule: safe ranges, enforced in code. Hardware limits → software constants; that's embedded engineering in one arrow.

② TEACH & DO — THE CORE (45')

- 25'** Guide pp. 30–61: robotic arm (30–35), grippers x2 (36–43 — “repeat for the other arm”), finish + cabling (44–61); route cables so full arm travel never pulls them taut.
- 10'** Feel-the-lever stations: joint vs. tip resistance; find the arm's natural range by hand, gently.
- 7'** Board: lever → torque → servo limit → ‘safe range in code’ arrow; write the numbers found by hand.
- 3'** Journal: label a sketch of the arm with joint, load, and where torque is highest.

QUICK START GUIDE — THIS SESSION

- Guide** same ZIP: arm pp. 30–35 → grippers pp. 36–43 (build TWO) → finish + cabling pp. 44–61; 3 rear D shafts, then ‘Done!’
- Screws** gripper → servo plate: M4×10 x4 + nuts x4 per arm (guide-verified); gripper shells x1+x2 boards + rubber band per arm
- Cables** STRAIGHT this year: left motor → EM1, right → EM2 — the Rover crossed them, this build does NOT · servo cables → S4 (left) / S3 (right) · short plastic ring first, then long

Watch out

- Students who remember Grade 8 will ‘helpfully’ cross the motor cables — the Ranger wires them STRAIGHT; crossed here means every turn runs backwards.
- Never force the arm past resistance — gear teeth strip silently and are not in the spare bag.
- The hand-found range (e.g. ≈0° floor to ≈100° up) becomes tomorrow's constants — write it down per robot.

+ THE 2-HOUR SESSION — THE OTHER 65' (2 TEACH & DO = THE 45' CORE ABOVE · 3 BREAK 5')

① RECAP & DEBUG

Feel it before explaining it. Unpowered, push the arm near the joint and then at the tip. That difference is torque, and it is a fact in the hands before it is a word on the board. Then find the natural range gently — where it stops moving freely is where the code will have to stop it tomorrow.

10' ④ EXTEND — TO SPEC

25'

full travel, no cable tension The arm moves through its range without pulling on any cable anywhere.

range measured per robot Safe floor and top angles found by hand and written down for *this* robot — not for the class, because they differ.

lever story, own sketch Joint, load, and where torque is highest, labelled by the team on their own drawing.

⑤ ENGINEERING LOG

The per-robot safe range with both numbers and a date, plus the labelled sketch. Tomorrow these two numbers become ARM_FLOOR and ARM_TOP, so a range measured carelessly today is a stripped gear next week and gears are not in the spare bag.

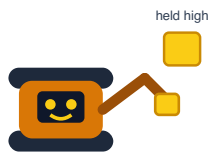
10' ⑥ DESIGN REVIEW

15'

Pairs explain torque from their own sketch. Criteria — does the explanation connect arm length to servo effort, and can they say why the range has to be per robot rather than copied from the team next door?

IF YOU FINISH EARLY **Break it** Find the arm position where the robot comes closest to tipping, without letting it tip. Mark that angle. It is a limit your code will have to respect and it appears in no guide.

• EXPECTED RESULT AT THE END OF THIS SESSION



the arm lifts and holds a block within its safe range

- ☒ Arm assembled; full travel without cable tension
- ☒ Team stated the lever/torque story on their own sketch
- ☒ Physical safe range measured and recorded per robot
- ☐ A per-robot safe range measured by hand and recorded with the robot's name
- ☐ The lever and torque story told from the team's own labelled sketch

First Arm Motions

Goal of these two hours: A guard that cannot be got round — proved by adversarial calls from someone trying to break it, not by reading the code.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled
built + range
measured

Rover add-on
box



Tablet + mBlock

Web

Classroom



USB cable

mBot2 box



Engineering
journal
why-one-guard
answer

Classroom

TEACH IT — THE CONCEPT

First powered motion: command the arm servo to angles from Python. Start ceremonial — lift, hold, lower, applause — then immediately install the engineering discipline: yesterday's hand-measured range becomes `ARM_FLOOR` and `ARM_TOP` constants, and every arm command goes through a guard function `arm_to(angle)` that clamps requests into the safe range. The robot should be physically unable to hurt itself through code.

This is the session's big idea, bigger than the arm: **defensive programming** — code that assumes mistakes will happen and contains them. Test it adversarially: call `arm_to(500)` on purpose; the guard should quietly deliver `ARM_TOP`. Fast pairs add speed control (move in small steps with sleeps) and discover smooth vs. jerky motion — aesthetics through timing, an echo of Grade 8's animation lesson.

② TEACH & DO — THE CORE (45')

- 12'** First powered lift/lower; find the servo command in autocomplete; map angle → physical position.
- 18'** Build `arm_to()` with clamping; constants from yesterday's measurements; adversarial tests.
- 10'** Choreography mini-challenge: a 3-pose salute using only `arm_to()` calls.
- 5'** Journal: why should limits live in ONE place (the guard) rather than at every call site?

PYTHON (MBLOCK → PYTHON MODE)

```
import mbot2, time

ARM_FLOOR = 0      # measured by hand,
Session 9
ARM_TOP    = 100

def arm_to(angle):
    # defensive: clamp into the safe range
    angle = max(ARM_FLOOR, min(ARM_TOP,
    angle))
    mbot2.servo_set(angle, 1)  # servo port
    1

arm_to(80)  # lift
time.sleep(1)
arm_to(500) # attack the guard -> goes to
100
arm_to(10)  # lower
```

Watch out

- Function names below follow mBlock's CyberPi Python API; minor names vary by version — the class rule is: trust the editor's autocomplete. The servo call especially varies — find it together in autocomplete.
- If the arm judders at range edges, back the constants off 5° — mechanical margins are wisdom.

+ THE 2-HOUR SESSION — THE OTHER 65' (② TEACH & DO = THE 45' CORE ABOVE · ③ BREAK 5')

① RECAP & DEBUG

Attack your own guard first. Negative angles, absurd angles, the exact boundary values. A guard that has not been attacked has not been tested, and the servo is the component that pays the difference.

10'

④ EXTEND — TO SPEC

25'

arm_to(500) is safe An absurd call delivers ARM_TOP quietly, with no juddering and no stall.

limits in one place Change the constant once and every call follows. If you have to edit two places, the guard is not a guard.

3-pose salute A choreography built from arm_to() calls only, with no raw servo command surviving anywhere in the file.

⑤ ENGINEERING LOG

The constants with their date and the robot they belong to, plus the adversarial test list: the call made, and what the guard did with it. This is the first appearance of defensive programming as a named idea, and it outlives the arm.

10'

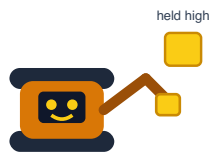
⑥ DESIGN REVIEW

15'

Pairs swap robots and attack another pair's guard. Criteria — did anything get through? A guard that survives a stranger actively trying to break it is the only kind worth having on a scored run.

IF YOU FINISH EARLY **Break it** Try to get an illegal angle past your guard by any route you can invent — through a variable, a calculation, a call inside a loop. Every hole you find today is a stripped gear you do not pay for later.

• EXPECTED RESULT AT THE END OF THIS SESSION



the arm lifts and holds a block within its safe range

- ☒ Arm moves to commanded angles via arm_to() only
- ☒ Guard proven by adversarial calls
- ☒ 3-pose salute runs smoothly
- ☐ Another pair failed to break the guard with adversarial calls
- ☐ The limit constant changed in one place and every call site followed

Pick & Place

Goal of these two hours: Rung 2 reached by every team, and the gripper's personality known — what it holds, and the condition under which it lets go.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled

Rover add-on
box



Tablet + mBlock
Web

Classroom



Foam blocks
NOT wooden blocks

Classroom



Masking tape
placement zones

Classroom



Engineering
journal

run log
Classroom



Color patches
the placement zone

Classroom

TEACH IT — THE CONCEPT

The challenge-ladder format: rungs of increasing difficulty, teams climb at their own pace, each rung demoed to the teacher before the next. Rung 1: grip and lift a foam block (grip force/angle tuning — too loose drops, too tight bends). Rung 2: lift, drive 30 cm, set down inside a taped zone (arm + motion coordination, and the discovery that driving shakes the payload — slower is surer). Rung 3: stack a second block on the first (precision: approach angle, release height — drop from too high and the tower falls).

Let the struggle happen — rung 3 is genuinely hard and that's its job. The debug discipline carries: one variable per attempt (grip angle OR approach speed OR release height), logged. Teams should end today knowing their gripper's personality: what it holds well, where it fumbles. That self-knowledge writes Session 12's failure log.

Watch out

- Foam blocks only — wooden ones strip servo gears on a bad grip.
- R2 zone generous (A5-size) today; precision tightens in the capstone.

② TEACH & DO — THE CORE (45')

- | | |
|-----|--|
| 8' | Ladder briefing + demo protocol; blocks and zones distributed. |
| 32' | Climb: grip→lift (R1), carry→place (R2), stack (R3); teacher demos at each rung. |
| 5' | Standings + journal: your highest rung and the single change that unlocked it. |

+ THE 2-HOUR SESSION — THE OTHER 65' (2 TEACH & DO = THE 45' CORE ABOVE · 3 BREAK 5')

① RECAP & DEBUG

Find the failure edges before climbing. Deliberately make the grip fail three ways: too loose, too tight, and approaching off-centre. Knowing all three by sight makes the ladder faster, because from then on a failed rung is diagnosed rather than simply retried.

⑤ ENGINEERING LOG

One row per attempt — rung, the single change made, outcome. The banned word today is "sometimes": a failure that happens sometimes has a condition attached, and finding that condition is the entire work of the hour.

10' ④ EXTEND — TO SPEC

25'

R2: carry and place Lift the block, drive 30 cm, and set it down inside the zone.

R3 attempted and logged The stack attempted with one-change-per-attempt discipline, whether or not it ever succeeds.

personality written down What this gripper holds well, and the condition under which it drops.

10' ⑥ DESIGN REVIEW

15'

Teams state their gripper's weakness. Criteria — is it stated as a condition ("drops when the block is approached more than about a centimetre off-centre") rather than as a frequency?

IF YOU FINISH EARLY **Constrain it** Complete rung 2 with the arm moving at half speed throughout. Slower is usually more reliable — find out whether that holds for your gripper, and by how much.

• EXPECTED RESULT AT THE END OF THIS SESSION



two blocks stacked inside the target zone

- ☒ Rung 2 (carry & place) reached by every team
- ☒ Rung 3 attempted with logged one-change attempts
- ☒ Gripper strengths/weaknesses written down
- ☐ Rung 2 reached with the carry-and-place complete
- ☐ The gripper's failure stated as a condition, never as "sometimes"

Reusable Arm Code

Goal of these two hours: A known reliability percentage for `pick()`, and a written decision about what to do with that number.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled
Rover add-on
box



Tablet + mBlock
Web
Classroom



Foam blocks
Classroom



Engineering
journal
failure log — 20
runs
Classroom

TEACH IT — THE CONCEPT

Yesterday's working sequences become a library: `pick()` (open, lower, close, lift), `place()` (lower, open, retreat, lift), `goto_cm(d)` (drive helper). The capstone mission will then read like English: `follow_line() ... pick() ... goto_cm(30) ... place()`. When top-level code reads like the mission description, the decomposition is right — that's the design smell to teach.

Second deliverable: the **failure log**. Run `pick()` twenty times; count successes; describe every failure precisely ('block rotated in grip when arm passed 60°', not 'it dropped it'). An 85% gripper is fine — an *unknown*-% gripper is a time bomb in a scored mission. Reliability engineering in one lunch-sized lesson: measure it, then decide whether to fix it or design around it (e.g. re-grip attempt on failure — which is Session 19's recovery topic, seeded today).

② TEACH & DO — THE CORE (45')

- 18'** Extract `pick()`, `place()`, `goto_cm()` from yesterday's code; parameters where useful.
- 15'** Failure log: 20 `pick()` runs, success count + precise failure notes.
- 8'** Library swap-test (the Session 2 ritual): another pair runs your `pick()` cold.
- 4'** Journal: your reliability %, and fix-it vs. design-around-it — which will you choose?

PYTHON (MBLOCK → PYTHON MODE)

```
def pick():
    grip_open()
    arm_to(ARM_FLOOR + 5)
    grip_close()
    time.sleep(0.3)      # let the grip
    settle
    arm_to(60)           # carry height

def place():
    arm_to(ARM_FLOOR + 10)
    grip_open()
    mbot2.straight(-5)   # retreat before
    lifting
    arm_to(60)
```

Watch out

- The sleep after `grip_close` matters — servos need a beat; its absence is the ghost bug of the week.
- Failure descriptions must name conditions; 'sometimes' is banned vocabulary.

+ THE 2-HOUR SESSION — THE OTHER 65' (② TEACH & DO = THE 45' CORE ABOVE · ③ BREAK 5')

① RECAP & DEBUG

Twenty runs, counted honestly, before improving anything. Whatever number comes out is the number the mission has to be designed around — and a team that does not know its own number will discover it during a scored run in Session 20 instead, which is the expensive way.

⑤ ENGINEERING LOG

The failure log with conditions named, and the reliability percentage stated plainly. An 85% gripper is entirely workable; an unknown-percentage gripper is a time bomb in a scored mission, and saying so now is what saves Session 18.

10' ④ EXTEND — TO SPEC

25'

swap test passed Another pair runs your pick() cold, with no explanation.

20 runs, precise failures Every failure described by its condition — "the block rotated in the grip as the arm passed 60°", not "it dropped it".

decision written Fix it, or design around it. Chosen, justified from the number, and written down.

10' ⑥ DESIGN REVIEW

15'

Pairs present their percentage and their decision. Criteria — does the decision follow from the number? Designing around a 60% gripper is engineering; hoping about it is not, and the two look identical until Demo Day.

IF YOU FINISH EARLY **Reliability** Run pick() twenty more times under a condition you have not tested: a different block position, or a lower battery. Two numbers under two conditions is far more useful than one number under one.

• EXPECTED RESULT AT THE END OF THIS SESSION

pick() runs: 20
success: 17 (85%)
fail: rotates past 60°
fail: edge grip on tilt
plan: slow lift 60–90°



the failure log — reliability measured, not assumed

- ☒ pick()/place()/goto_cm() pass the swap-test
- ☒ 20-run failure log with precise descriptions
- ☒ A reliability decision made and written down
- ☐ A reliability percentage measured over 20 runs and stated plainly
- ☐ A fix-it or design-around-it decision written and justified by the number

Navigate + Manipulate *

Goal of these two hours: The pipeline measured for repeatability, with both seams named and their failure modes characterised.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled
Rover add-on
box



Tablet + mBlock
Web
Classroom



Line-following
map
mBot2 box



Color patches
the pick marker
Classroom



Foam blocks
Classroom



Masking tape
delivery zone
Classroom



Engineering
journal
Classroom

TEACH IT — THE CONCEPT

Integration day — the capstone in miniature, and deliberately so: line following (Session 6) + color-marker detection (Grade 7, Session 16's idea, now in Python) + arm library (Session 12) chained into one mission. The new engineering content is the *seams*: stopping accurately AT the marker (momentum overshoots — the two-speed approach returns yet again), and positioning the arm relative to a block whose location is only as precise as the stop was. Errors compound down a pipeline; students meet that truth physically today.

Run it as a milestone session: the pipeline working once, even slowly and inelegantly, is the win. Teams that get a clean run should immediately answer the harder question: how repeatable? Three runs in a row is the real bar — and the gap between 'worked once' and 'works reliably' is the theme of the entire build-sprint unit ahead.

② TEACH & DO — THE CORE (45')

- 8'** Whiteboard the pipeline; identify the two seams (stop accuracy, arm alignment).
- 30'** Build and iterate; markers and blocks on the standard course; one-change discipline at the seams.
- 7'** Milestone attempts: pipeline end-to-end; then repeatability count (n clean of 3).

PYTHON (MBLOCK → PYTHON MODE)

```
while True:
    err =
    cyberpi.quad_rgb_sensor.get_offset_track()
    if
    cyberpi.quad_rgb_sensor.is_color('green'):
        mbot2.EM_stop()
        break
    steer = err * GAIN
    mbot2.drive_speed(BASE + steer, BASE -
    steer)

pick()
mbot2.turn(180)
goto_cm(40)
place()
celebrate()      # Session 2, still earning
its keep
```

Watch out

- Buffer session — a struggling class can pre-place the block at a fixed, known stop point.
- Slow the BASE speed near expected markers; detection at speed is flaky and that's a real sensor lesson.

+ THE 2-HOUR SESSION — THE OTHER 65' (② TEACH & DO = THE 45' CORE ABOVE · ③ BREAK 5')

① RECAP & DEBUG

Name the seams before building anything. Whiteboard the pipeline and mark the two joins — stopping accurately at the marker, and aligning the arm to the block — then predict which will fail more often. Nearly everything that goes wrong today lives at a seam rather than inside a stage, and predicting which one focuses the hour.

⑤ ENGINEERING LOG

A run ledger recording the phase reached on every attempt, plus the two seam descriptions. "Worked once" and "works" are different claims about the world, and the ledger is the only thing that tells them apart.

10' ④ EXTEND — TO SPEC

end-to-end once The full pipeline completed, however slowly and however inelegantly.

n of 3 measured Repeatability counted and recorded, whatever the number turns out to be. Zero of three is data.

both seams characterised Each seam's failure mode described with the condition that triggers it.

10' ⑥ DESIGN REVIEW

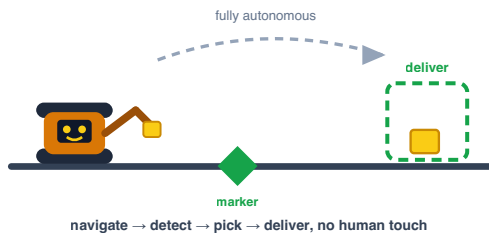
Teams compare seam failures across the class. Criteria — is the failure attributed to a specific seam with evidence behind it, rather than to the mission simply being hard?

25'

15'

IF YOU FINISH EARLY **Break it** Stop the robot 3 cm short of the marker on purpose and see whether the arm still succeeds. That margin is your real tolerance, and it tells you which of the two seams deserves Session 16.

• EXPECTED RESULT AT THE END OF THIS SESSION



- ☒ Full pipeline ran end-to-end at least once
- ☒ Both seams named with their observed failure mode
- ☒ Repeatability measured (n of 3), whatever the n
- ☐ Repeatability measured as n of 3 and recorded honestly
- ☐ Both seams described with the conditions that trigger them

Program Structure

Goal of these two hours: A state diagram that survives two failure scenarios on paper, and restructured code whose behaviour still matches Session 13.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled
Rover add-on
box



Tablet + mBlock
Web
Classroom



Engineering
journal
state diagram —
deliverable
Classroom



Printed cards
trace scenarios
Classroom



Craft kit (card,
markers, tape)
large paper for the
diagram
Classroom

TEACH IT — THE CONCEPT

Session 13's pipeline was a straight script — fine until something fails mid-mission and the program can't express 'go back and retry'. Introduce the **state machine** pattern: a state variable (SEARCHING / APPROACHING / PICKING / DELIVERING / DONE), a main loop that reads sensors and, depending on the current state, acts and decides transitions. It's Grade 8's mood variable wearing a hard hat — say exactly that; the recognition lands.

Before running: **paper trace**. One teammate is the CPU: finger on the current state, walk through a scenario ('marker seen → state becomes PICKING'), including the nasty ones ('what if the grip fails in PICKING?' → they'll want a retry transition — let them invent it). Tracing catches design holes in minutes that debugging would find in hours; it is also precisely how they'll explain their code in the capstone review.

PYTHON (MBLOCK → PYTHON MODE)

```
state = 'SEARCHING'

while state != 'DONE':
    if state == 'SEARCHING':
        follow_line_step()
        if see_marker():
            state = 'PICKING'
    elif state == 'PICKING':
        if pick_ok():
            state = 'DELIVERING'
        else:
            state = 'SEARCHING' # retry
    path!
    elif state == 'DELIVERING':
        goto_cm(40); place()
        state = 'DONE'
```

② TEACH & DO — THE CORE (45')

- 12'** Board the state diagram for the S13 mission; agree states and transitions as a class.
- 10'** Paper trace in pairs, including two failure scenarios; patch the diagram where it breaks.
- 18'** Restructure the code to the state pattern; behavior should match S13's (refactor discipline!).
- 5'** Journal: your state diagram, photographed or redrawn.

Watch out

- States are NOUNS-in-progress, transitions are events — keep the diagram honest.
- If the trace can't answer 'what happens if X', the code can't either; fix the diagram first.

+ THE 2-HOUR SESSION — THE OTHER 65' (② TEACH & DO = THE 45' CORE ABOVE · ③ BREAK 5')

① RECAP & DEBUG

Be the CPU. One partner puts a finger on the current state and walks a scenario aloud while the other reads the diagram. Do the pleasant scenario first, then the nasty one — the grip fails during PICKING. If the diagram has no answer, the code will not have one either, and the cheapest place to fix that is on paper.

⑤ ENGINEERING LOG

The state diagram, photographed or redrawn, with states as nouns-in-progress and transitions as events. Plus the two traces and the patches they forced — the patches are the useful part, because they are the failures caught before they cost a run.

10' ④ EXTEND — TO SPEC

25'

retry path exists At least one failure transition that returns the machine to a working state instead of stopping.

2 scenarios survived Two failure traces walked end to end without the diagram breaking.

behaviour unchanged The restructured code matches Session 13's behaviour. This is a refactor, and refactors are checkable.

10' ⑥ DESIGN REVIEW

15'

Pairs trace another pair's diagram using a scenario of their own choosing. Criteria — did the diagram answer, or did the author have to speak? An author who has to explain their own state machine has found a bug in it.

IF YOU FINISH EARLY **Bank it** Add one more failure transition to the diagram — the one you are sure will never happen — and draw it in a different colour. In Session 18 you will find out whether you were right.

• EXPECTED RESULT AT THE END OF THIS SESSION

```
state = 'SEARCHING'
while state != 'DONE':
    if state=='SEARCHING':
        ...
    elif state=='PICKING':
        ...retry on fail...
```

the mission as a state machine — retry paths included

```
d = 34 cm
d = 27 cm
d = 12 cm
STOP
```

- ☒ State diagram covers the mission incl. one retry path
- ☒ Paper trace survived two failure scenarios
- ☒ Restructured code matches S13 behavior
- ☐ Two failure scenarios traced on paper before any code was changed
- ☐ Restructured behaviour still matches Session 13's, measured not assumed

Design Sprint

Goal of these two hours: A mission specified in numbers, with the cut line decided while nobody is yet under any pressure.

YOU NEED — PIECES & MATERIALS



Engineering journal
spec: 4 artifacts

Classroom



Ruler / tape measure
course dimensions

Classroom



Printed cards
rubric + example missions

Classroom



Ranger, assembled
feasibility checks only

Rover add-on box



Craft kit (card, markers, tape)

Classroom

TEACH IT — THE CONCEPT

The brief, in full: design an autonomous mission simulating a real job — warehouse sorting, rescue supply delivery, harvesting, waste collection, or the team's own pitch. The robot must navigate (line or programmed path), detect a target (marker/color), manipulate (pick and deliver at least one object) — fully autonomously. Demo Day (Session 20): scored runs plus a 5-minute pitch that must include one honest failure story.

Today produces four artifacts, in order: **problem statement** (what real job, why it matters — the pitch's backbone); **success criteria** (measurable: 'delivers 2 blocks to zones in <90 s, 2 of 3 runs'); **course sketch** (top-down, dimensions, line path, marker and zone positions); **task breakdown** mapped onto Sessions 1–14 (every task should point at a session where its skill was built — anything pointing nowhere is scope risk, cut it or flag it). The sign-off gate is strictest of all three years: ambition is celebrated, unscoped ambition is returned for revision. 'Impressive and finished' beats 'amazing and imaginary' — write it on the board.

Watch out

- Ban criteria without numbers — 'works well' is not a criterion.
- Require a written cut line: the feature they'll drop first if time runs short. Deciding it now removes the mid-sprint agony.

② TEACH & DO — THE CORE (45')

- | | |
|-----|---|
| 8' | Brief + rubric + example missions pitched fast (steal-able starting points). |
| 12' | Problem statement + success criteria drafted; criteria must contain numbers. |
| 18' | Course sketch + task breakdown with session references. |
| 7' | Sign-off interviews: scope probe ('which task is riskiest? what's your cut line?'). |

+ THE 2-HOUR SESSION — THE OTHER 65' (② TEACH & DO = THE 45' CORE ABOVE · ③ BREAK 5')

① RECAP & DEBUG

Attack the scope. Each team hands its draft criteria to another team whose only job is to find the vaguest one. "Works well" and "delivers reliably" die here, cheaply, instead of dying at the sign-off gate or — far worse — on Demo Day in front of an audience.

10' ④ EXTEND — TO SPEC

25'

criteria contain numbers How many, how fast, how many runs out of how many. Every criterion measurable by someone who was not there.

every task traced Each task maps to the session that built the capability it depends on.

cut line written The feature to be dropped first if time runs short — decided today, in writing, while it is still an abstract decision.

⑤ ENGINEERING LOG

The four artifacts — problem statement, numeric success criteria, course sketch with dimensions, task breakdown with session references — plus the cut line. The cut line is the page that matters at the end of Session 18, and deciding it now removes the argument.

10' ⑥ DESIGN REVIEW

15'

Sign-off interviews with a scope probe. Criteria — which task is riskiest, and does the cut line remove the riskiest thing, or merely the thing that was easiest to give up?

IF YOU FINISH EARLY **Break it** Give your success criteria to another team and ask them to score a run they did not watch, from the criteria alone. If they cannot, the criteria are not measurable yet, whatever numbers they contain.

• EXPECTED RESULT AT THE END OF THIS SESSION

job: pharmacy delivery
succeed: 2 zones < 90 s
course: fig. 8 + 2 markers
riskiest: double pick
cut line: 2nd delivery



the signed mission spec — numbers, sketch, cut line

- ☒ All four artifacts complete; criteria numeric
- ☒ Every task maps to a source session
- ☒ Cut line written; sign-off passed
- ☐ Every success criterion contains a number a stranger could check
- ☐ The cut line written down before any time pressure exists

Build Sprint I *

Goal of these two hours: Navigation that arrives reliably three times running, on a course the team can rebuild exactly from its own photograph.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled

Rover add-on
box



Tablet + mBlock
Web

Classroom



Line-following
map
or taped path

mBot2 box



Masking tape
course build

Classroom



Foam blocks

Classroom



Stopwatch

Classroom



Phone (video/
angle)

photograph the
layout + tape
measure

Classroom



Engineering
journal

Classroom

TEACH IT — THE CONCEPT

Sprint rules, borrowed from real teams: a 2-minute standup at the top (each team: done / doing / blocked — blockers get teacher attention first), then heads-down build, then a milestone check at the end. Today's milestone for everyone regardless of mission: **navigation end-to-end** — the robot travels its full course path and stops where the mission needs it, three times in a row.

Hold the line on order: teams itching to build the arm show first that the robot can reliably *arrive*. Navigation is the foundation every later behavior stands on; a wobbly foundation makes every arm bug ambiguous ('did the grip fail or did we stop 4 cm short?'). Reliability before features — it's the sprint's only commandment.

Watch out

- Buffer session: teams behind on P-control tuning catch up here — that IS their navigation work.
- Courses on the floor get shuffled between sessions; photograph the layout with a tape measure in frame.

② TEACH & DO — THE CORE (45')

- 5'** Standup circle: done / doing / blocked; blockers triaged.
- 33'** Build: course laid out to the sketch's dimensions; navigation tuned on the real course.
- 7'** Milestone check: 3 consecutive end-to-end navigation runs, witnessed and logged.

+ THE 2-HOUR SESSION — THE OTHER 65' (② TEACH & DO = THE 45' CORE ABOVE · ③ BREAK 5')

① RECAP & DEBUG

Standup, and time it visibly. Done, doing, blocked — two minutes total, with blockers going to the teacher first. A standup that runs to six minutes has become a status meeting and helps nobody; holding the clock is the whole discipline.

⑤ ENGINEERING LOG

The run ledger and the course photograph with dimensions. Floors get shuffled between sessions, and a course that cannot be rebuilt exactly makes every measurement taken on it incomparable with the next one.

10' ④ EXTEND — TO SPEC

3 consecutive clean runs Witnessed and logged on the team's own course — three in a row, not the best three of six.

course rebuilt from photo The layout reproduced from a photograph taken with a tape measure in frame.

arrives before anything else No arm work until navigation is reliable, so that later arm bugs are unambiguous.

10' ⑥ DESIGN REVIEW

Milestone check with a witness from another team. Criteria — three consecutive runs, and was the course rebuilt to the sketch's dimensions rather than to roughly the right shape?

25'

15'

IF YOU FINISH EARLY **Reliability** Run navigation five more times and record the *spread* of where the robot stops, not merely whether it arrived. That spread is what your arm will have to tolerate in Session 18.

• **EXPECTED RESULT AT THE END OF THIS SESSION**



2-minute standup: done · doing · blocked

- ☒ 3 consecutive clean navigation runs on the team's own course
- ☒ Standup done in under 2 minutes with a real blocked-list
- ☒ Course layout photographed for exact rebuild
- ☐ Three consecutive clean navigation runs witnessed by another team
- ☐ The course rebuilt from a photograph with dimensions visible

Build Sprint II + Code Review

Goal of these two hours: Code that survives a stranger reading it against the rubric's own checklist, with at least one suggestion applied the same hour.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled
Rover add-on
box



Tablet + mBlock
Web
Classroom



Printed cards
review checklist x2
per team
Classroom



Foam blocks
Classroom



Engineering
journal
2+2 review notes
Classroom



Laptop + mBlock
Classroom

TEACH IT — THE CONCEPT

Build continues (arm behaviors onto the navigation spine), but the hour's signature event is the **cross-team code review** — the professional practice, scaled down. Pairs of teams swap laptops and read each other's code against a four-line checklist: *functions with clear names?* · *constants instead of magic numbers?* · *comments that say why?* · *no copy-pasted near-duplicates?* Reviewers write two 'strengths' and two 'suggestions' — the discipline of kind precision.

Frame it before anyone reads a line: review is about the code, not the coder; every suggestion is a gift that saves a future debugging hour. The checklist is the capstone rubric's code-quality section verbatim — so today is a free mock exam. Teams then get sprint time to act on what review found; acting on feedback is the half of the skill that classrooms usually skip.

Watch out

- Model one review comment style first: 'pick() is very readable' / 'GAIN=0.5 appears 3x — constant?'
- A team 'too busy to review' is the team that most needs it — the checklist takes 12 minutes, hold firm.

② TEACH & DO — THE CORE (45')

- 5'** Standup + review protocol and the four-line checklist.
- 12'** Cross-team review: read, checklist, write 2 strengths + 2 suggestions.
- 23'** Sprint: apply at least one review suggestion, then continue manipulation build.
- 5'** Milestone pulse: where is each team vs. the plan? Cut lines re-checked aloud.

+ THE 2-HOUR SESSION — THE OTHER 65' (2 TEACH & DO = THE 45' CORE ABOVE · 3 BREAK 5')

① RECAP & DEBUG

Set the tone before anyone reads a line: review is about the code, not the coder, and every suggestion is a gift that saves someone a future debugging hour. Model one comment of each kind aloud — a strength and a suggestion — so that nobody's first attempt at this comes out as a personal remark.

10' ④ EXTEND — TO SPEC

25'

2 strengths, 2 suggestions Written, specific, and pointed at named functions or lines rather than at the file in general.

one applied this hour A suggestion acted on before the session ends — not noted down for a later that never arrives.

all 4 checklist lines Clear function names, no repeated magic numbers, comments saying why, dead code removed.

⑤ ENGINEERING LOG

The review received and the review given, plus the change made in response. The checklist is the capstone rubric's code-quality section word for word, so this hour is a free mock exam and this page is the mark sheet.

10' ⑥ DESIGN REVIEW

15'

Teams report what they actually changed. Criteria — was the suggestion applied or merely acknowledged? The team that says it is too busy to review is reliably the team that most needs it, and the checklist takes twelve minutes.

IF YOU FINISH EARLY **Teach it** Review a second team's code, not only the one you were paired with. The second review is faster and sharper than the first, and it is the same twelve minutes and the same checklist.

• EXPECTED RESULT AT THE END OF THIS SESSION



2-minute standup: done · doing · blocked

- ☒ Review exchanged: 2+2 written comments per team
- ☒ At least one suggestion applied the same hour
- ☒ Manipulation visibly progressed on the mission course
- ☐ Two strengths and two suggestions written, and the same received
- ☐ At least one suggestion applied within the hour, not deferred

Build Sprint III

Goal of these two hours: A phase-level run log that turns frustration into data, and one seam or drift fix made through a single change.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled

Rover add-on
box



Tablet + mBlock
Web

Classroom



Foam blocks

Classroom



Engineering
journal

run log on the wall

Classroom



mBot2 Shield
log battery level too

mBot2 box



Craft kit (card,
markers, tape)

wall run-log sheets

Classroom



Stopwatch

Classroom

TEACH IT — THE CONCEPT

Integration week peaks: manipulation joins navigation and full mission attempts begin. Expect the seam bugs from Session 13 at mission scale — stop accuracy vs. arm alignment — plus the new one: **cumulative drift** over a longer course (small navigation errors sum; missions that pick early and deliver late feel it most). The state-machine structure from Session 14 earns its keep now: teams with clean states can add a re-align step to one state without touching the rest.

Keep a visible run log per team on the wall: attempt number, phases completed, failure point. The log turns frustration into data — ‘we fail at delivery 3 of 4 times’ is actionable; ‘it keeps breaking’ is despair. It also quietly writes the failure-story half of the pitch. Teams at their cut line make the cut today, without shame — point at the board from Session 15.

Watch out

- Enforce the log — memory lies under sprint pressure, paper doesn’t.
- Battery sag changes stop distance late in the hour; note battery level in the log too (Grade 7 wisdom, still true).

② TEACH & DO — THE CORE (45’)

- 5’** Standup; run-log sheets posted per team.
- 33’** Full mission attempts; every attempt logged; fixes between runs follow one-change discipline.
- 7’** Milestone: at least one full-mission success, or the cut executed and the reduced mission succeeding.

+ THE 2-HOUR SESSION — THE OTHER 65' (2 TEACH & DO = THE 45' CORE ABOVE · 3 BREAK 5')

① RECAP & DEBUG

Read the log before touching the robot. Whatever the wall sheet says fails most is where the hour goes. "We fail at delivery three times out of four" is a work item; "it keeps breaking" is a mood — and under sprint pressure the log is the only reliable difference between the two.

10'

④ EXTEND — TO SPEC

25'

full mission once Original scope or the post-cut scope. Executing the cut counts as success, and pretending the cut was not needed does not.

phase-level logging Every attempt records the phase it reached, not merely pass or fail.

one-change fixes only Every fix between runs is a single logged change, so the log stays readable as evidence.

⑤ ENGINEERING LOG

The wall run log — attempt, phases completed, failure point, battery level. Battery sag changes stop distance late in the hour; that was Grade 7 wisdom and it is still true two years and two robots later.

10'

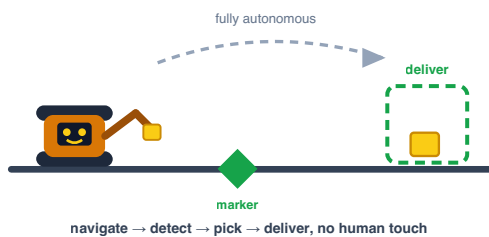
⑥ DESIGN REVIEW

15'

Teams read their own failure distribution aloud. Criteria — does the log identify one dominant failure point, and is the next action aimed at that point rather than at whichever failure was most annoying to watch?

IF YOU FINISH EARLY **Reliability** Run the mission three more times changing absolutely nothing. Resist fixing. Three clean runs of unchanged code is the only real evidence that your last fix worked, and it is the step teams skip most often.

• EXPECTED RESULT AT THE END OF THIS SESSION



- ☒ Full mission completed at least once (original or post-cut scope)
- ☒ Run log with phase-level failure data
- ☒ One drift/seam fix implemented via a single state change
- ☐ The dominant failure point identified from the log rather than from memory
- ☐ Every between-run fix was a single logged change

Rehearsal & Pitch Prep *

Goal of these two hours: One recovery behaviour that demonstrably triggers, a configuration frozen in writing, and a pitch that fits five minutes twice running.

YOU NEED — PIECES & MATERIALS



**Ranger,
assembled**
Rover add-on
box



**Tablet + mBlock
Web**
Classroom



Stopwatch
strict 5:00
Classroom



**Engineering
journal**
frozen race config
Classroom



Printed cards
pitch structure card
Classroom



USB cable
chargers on before
the session ends
mBot2 box

TEACH IT — THE CONCEPT

Two workstreams. **Robot:** rehearse under Demo-Day conditions (official start, audience noise simulated, two attempts) and add cheap recovery where the run log points — the classics: re-grip once if pick fails (Session 12's seed sprouts), re-find line if lost (Grade 7's search behavior returns), timeout-and-skip rather than freeze forever. Then freeze the configuration — the Grade 7 race-config ritual, now second nature.

Pitch (5 minutes, strict): the problem (30 s) → the solution & how it works (2') → live demo (90 s) → **one honest failure story** and what it taught (1'). The failure story is mandatory and rubric-weighted — it's where engineering maturity shows, and after three years of debug discipline every team has a good one. Rehearse with a timer; cutting to fit five minutes is itself the exercise. Assign demo roles: driver (starts the run), narrator (talks over it), catcher (handles the robot if chaos wins).

Watch out

- Buffer session: recovery can shrink to timeout-and-skip only — but the freeze and the timed rehearsal are untouchable.
- The failure story must end on the lesson, not the excuse — coach the last sentence.

② TEACH & DO — THE CORE (45')

- | | |
|-----|--|
| 18' | Recovery pass: implement the top recovery the run log justifies; verify it triggers correctly. |
| 8' | Freeze: race configuration written, code committed, no edits after this line. |
| 15' | Pitch rehearsal x2 with timer; peers give one keep / one cut note. |
| 4' | Demo-day logistics: order drawn, roles assigned, batteries on chargers. |

+ THE 2-HOUR SESSION — THE OTHER 65' (② TEACH & DO = THE 45' CORE ABOVE · ③ BREAK 5')

① RECAP & DEBUG

Let the run log choose the recovery. Do not build the one that sounds most impressive — build the one the failure data justifies: re-grip once, re-find the line, or time out and skip. The log names which, and building the wrong one spends a whole session solving a problem this robot does not have.

⑤ ENGINEERING LOG

The frozen configuration and the pitch script with timings per section. The failure story is rubric-weighted, so its last sentence is worth drafting rather than improvising — it has to end on the lesson, never on the excuse.

10' ④ EXTEND — TO SPEC

recovery triggers on demand Cause the failure deliberately and watch the recovery fire. A recovery never seen to trigger is a recovery nobody should count on.

frozen, in writing The race configuration written out in full, with no edits after that line.

5:00 twice, all four parts Problem, solution, live demo, and the honest failure story — timed, twice.

10' ⑥ DESIGN REVIEW

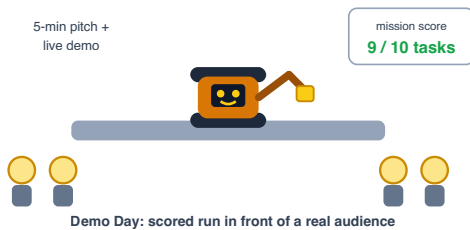
Peers give one keep note and one cut note per pitch. Criteria — does the failure story end on what was learned, and did the live demo fit inside its ninety seconds without anyone having to rush?

25'

15'

IF YOU FINISH EARLY **Teach it** Tell your failure story to a team on a different mission and ask them one question: did that teach you something you could use? If not, it is still an excuse rather than a lesson, and the last sentence needs rewriting.

• EXPECTED RESULT AT THE END OF THIS SESSION



- ☒ One recovery behavior demonstrably triggers and works
- ☒ Configuration frozen in writing
- ☒ Pitch fits 5:00 with all four parts, twice
- ☐ The recovery fires when the failure is caused deliberately
- ☐ The pitch fits 5:00 twice, with all four parts present both times

Demo Day

Goal of these two hours: Scored runs against each team's own signed criteria, pitches with the failure story told, and the three-year question answered again.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled
frozen config

Rover add-on
box



Tablet + mBlock
Web

Classroom



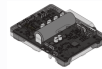
Foam blocks

Classroom



Stopwatch
+ scoreboard

Classroom



mBot2 Shield
charged overnight

mBot2 box



Phone (video/
angle)
course rebuild
photos

Classroom



Printed cards
award certificates

Classroom



Engineering
journal

Classroom

TEACH IT — THE CONCEPT

Run it as the event three years earned: audience seated (invite classes, teachers, parents if possible), visible scoreboard (phase points per team's own success criteria + small style bonus), two attempts each with pit time between, pitches delivered from the front. The teacher's job today is master of ceremonies — the students carry the content, and they can.

Close the arc deliberately. The retrospective ritual: the Session 1/ Grade 7 question projected ('what is a robot?') — let them answer it again, now with three years of hands. Then the pathways talk, concrete and short: what they built maps to real fields — control engineering (their P-controller), ML engineering (their mirror), embedded software (their guard functions), robotics competitions, and the kit's 30+ module ecosystem for a robotics club afterlife. End on the honest sentence: they didn't learn about robotics — they did robotics, sixty hours of it.

Watch out

- Rebuild courses from the tape-measure photos — 'the course moved' is a heartbreak with a known cure.
- Score against each team's OWN success criteria — missions differ; fairness lives in the spec they signed.

② TEACH & DO — THE CORE (45')

- | | |
|-----|--|
| 8' | Setup: courses rebuilt from photos, scoreboard up, order posted, audience seated. |
| 30' | The event: per team — pitch (5' incl. live demo) with scored attempts; pit repairs allowed between rounds. |
| 5' | Awards: precision, engineering, comeback, audience choice. |
| 2' | Retrospective question + pathways + the closing sentence. |

+ THE 2-HOUR SESSION — THE OTHER 65' (② TEACH & DO = THE 45' CORE ABOVE · ③ BREAK 5')

① RECAP & DEBUG

Rebuild the courses from the photographs, with the tape measure, before anything else happens. "The course moved" is a heartbreak with a known cure, and the cure is fifteen minutes of measuring rather than an argument about scoring in front of an audience.

⑤ ENGINEERING LOG

The scoreboard archived, and each student's final reflection set against the Grade 8 Session 1 prediction of which skill they would reuse most. Three years of journals close on this page, and the comparison is worth the two minutes it costs.

10' ④ EXTEND — TO SPEC

2 scored attempts Against the team's own signed success criteria — the missions differ, so fairness lives in the spec each team signed in Session 15.

5-minute pitch, 4 parts Including the failure story, which is rubric-weighted and applauded rather than endured.

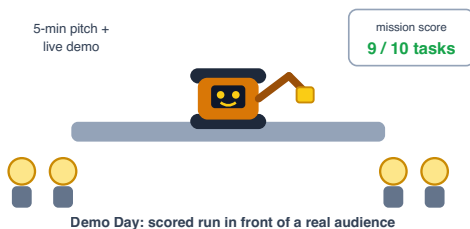
answer it again The Grade 7 Session 1 question — what is a robot? — answered with three years of hands behind it.

10' ⑥ DESIGN REVIEW

15' The retrospective. Criteria — is the new answer to "what is a robot?" richer than the Grade 7 one in a way the student can actually point at? Then the pathways talk, concrete and short: control engineering, robotics, data — and what they built maps to each.

IF YOU FINISH EARLY **Bank it** Write the note the next Grade 9 class should read before their own design sprint: the single scoping decision you would make differently. Three years of journals end here — leave something behind that is actually usable.

• EXPECTED RESULT AT THE END OF THIS SESSION



- ☒ Every team: two scored attempts + a complete 5-minute pitch
- ☒ Failure stories told and applauded
- ☒ Scoreboard archived; three-year arc closed
- ☐ Every team scored against its own signed success criteria
- ☐ The Grade 7 question answered again, with evidence of how the answer changed